MPA: Lightweight and Updatable Integrity Auditing for Decentralized Storage Using Merkle Trees and Polynomial Commitments

Yongliang Xu, Hang Cheng, Jingyu Zheng, Xinpeng Zhang, *Senior Member, IEEE*, and Huaxiong Wang

***Abstract*—With the growing demand for outsourcing data to decentralized storage systems, ensuring the integrity of outsourced data becomes a critical challenge. Existing auditing schemes, however, often assume single-copy or centralized models, and suffer from inefficiency, lack of public verifiability, or poor scalability in multi-replica settings. To address these limitations, we propose MPA, a lightweight and publicly verifiable auditing scheme tailored for multi-copy cloud storage. By integrating polynomial commitment schemes with Merkle trees, our design achieves efficient block-level integrity verification while enabling dynamic updates. To mitigate collusion between cloud service providers, each data copy is uniquely encrypted, and the audit process supports simultaneous verification across multiple providers. Furthermore, we introduce an optimized batch auditing mechanism that allows the verifier to aggregate proofs across different files and providers, reducing both computation and communication overhead. To enhance audit transparency and unpredictability, we adopt a blockchain-assisted challenge generation protocol based on commit-and-reveal randomness. Theoretical analysis and performance evaluation demonstrate that MPA achieves strong security guarantees under standard assumptions, while significantly outperforming existing solutions in terms of efficiency and scalability.**

***Index Terms*—Data integrity auditing, Merkle trees, polynomial commitment, blockchain-based randomness, decentralized storage.**

I. Introduction

With the continuous growth of global data volume, ensuring data integrity and security becomes increasingly important [1], [2]. According to the latest forecast by the international data corporation (IDC), the global datasphere is expected to expand from approximately 143 ZB in 2024 to 291 ZB by 2028, with a compound annual growth rate (CAGR) of around 18%, driven by digital transformation, IoT proliferation, and the adoption of generative AI applications [3]. Meanwhile, the global cloud storage market is also projected to grow significantly, from USD 145.23 billion in 2025 to USD 425.76 billion by 2030, at a CAGR of 24% [4]. Although centralized cloud storage remains mainstream, it is increasingly criticized for issues regarding scalability, cost-efficiency, and security. Decentralized storage solutions such as Filecoin, IPFS, and Arweave have emerged as promising alternatives. Benzinga predicts that the adoption of Filecoin will continue to rise, with its token (FIL) price expected to grow from an average of USD 4.10 in 2025 to USD 13.26 by 2028, reflecting increasing market confidence in decentralized storage technologies [5].

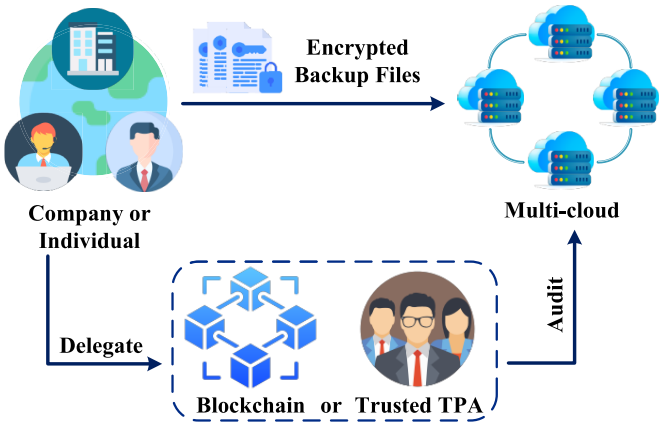


Fig. 1. Application scenario of MPA in multi-cloud auditing.

Fig. 1 illustrates a typical application scenario of the data auditing framework. A data owner, such as a company or individual, stores encrypted backup files across multiple cloud storage providers. To ensure data availability and integrity, the owner can delegate the audit task to either a trusted third-party auditor (TPA) or a decentralized blockchain-based verifier. However, most existing integrity auditing schemes [6], [7] are based on provable data possession (PDP) and proofs of retrievability (PoR), which were originally designed for single-copy, centralized environments. These schemes struggle to scale in multi-copy, decentralized contexts. Recent blockchain-assisted auditing schemes [8], [9] offer improvements but often suffer from large proof sizes or expensive pairing computations, limiting their practicality.

Received 15 May 2025; revised 8 January 2026; accepted 9 February 2026. Date of publication 13 February 2026; date of current version 19 February 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2025YFA1016901 and in part by the National Natural Science Foundation of China under Grant 62172098 and Grant 62471141. The associate editor coordinating the review of this article and approving it for publication was Dr. Haomiao Yang. *(Corresponding author: Hang Cheng.)*

Yongliang Xu, Hang Cheng, and Jingyu Zheng are with the School of Mathematics and Statistics, Fuzhou University, Fuzhou 350108, China (e-mail: 230310014@fzu.edu.cn; hcheng@fzu.edu.cn; 240320070@fzu.edu.cn).

Xinpeng Zhang is with the School of Computer Science, Fudan University, Shanghai 200433, China (e-mail: zhangxinpeng@fudan.edu.cn).

Huaxiong Wang is with the School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore 639798 (e-mail: hxwang@ntu.edu.sg).

Digital Object Identifier 10.1109/TIFS.2026.3664004

To address these challenges, we propose MPA, a lightweight and updatable integrity auditing scheme tailored for decentralized multi-copy storage. MPA combines polynomial commitments and Merkle tree structures to support efficient block-level verification, and dynamic updates. By uniquely encrypting each data copy, MPA prevents collusion among cloud service providers (CSPs). Furthermore, a blockchain-assisted commit-and-reveal randomness protocol ensures transparent and unpredictable challenge generation.

The main contributions of this paper are summarized as follows:

- We propose MPA, the first integrity auditing scheme that tightly integrates polynomial commitments with Merkle tree authentication. This combination ensures efficient, updatable and block-level integrity auditing tailored for decentralized multi-copy cloud storage.
- We introduce a blockchain-assisted commit-and-reveal protocol for transparent blockchain auditing, which ensures unpredictability in challenge generation, preventing precomputed proof attacks and enhancing audit fairness.
- We extend MPA to support multi-file and multi-provider batch auditing. By aggregating proofs across multiple files and CSPs into a single pairing check, our scheme significantly reduces both computation and communication overhead for large-scale audits.
- We rigorously prove MPA's security properties and formally quantify its probabilistic detection and randomness guarantees to establish concrete security bounds. Unlike prior designs that rely on server-side aggregation, our scheme delegates aggregation to the verifier, enabling trustless batch verification across multiple files and providers. This eliminates trust assumptions on the CSP and prevents aggregation forgery. Evaluations against state-of-the-art schemes demonstrate MPA's superior computational efficiency and scalability.

Section II reviews related work, followed by preliminaries in Section III. Section IV formalizes the system and security models. Section V details the proposed MPA protocol, which is analyzed in Section VI. Finally, Section VII presents the performance evaluation, and Section VIII concludes the paper.

II. Related Work

Auditing the integrity of outsourced data has been widely explored. Classic schemes such as PDP [10] and PoR [11] laid the foundation for verifying data correctness without retrieving full files. Later extensions like CPOR [12] and HAIL [13] provided multi-server support and fault tolerance, yet were designed primarily for centralized architectures.

A. Blockchain-Based Decentralized Auditing

With the emergence of decentralized storage, ensuring tamper-resistant and publicly verifiable integrity remains a critical challenge. Zhang et al. [14] proposed a blockchain-based dynamic auditing framework using time-encapsulated tags to prevent reuse attacks. Similarly, DWare [15] introduced a cost-efficient decentralized framework with adaptive middleware to balance performance and redundancy. To address auditor trust, Xu et al. [16] combined biometric identity with threshold secret sharing for fuzzy identity-based verification. Specific scenarios have also been addressed: Song et al. [17] and Yu et al. [18] developed auditing schemes supporting deduplication in multi-tenant settings. For IoT environments, Liang et al. [19] combined smart contracts with IPFS, while Hou et al. [20] introduced CCMR, utilizing CC-MHT and multi-signature aggregation to reduce redundancy. Additionally, Miao et al. [21] focused on structured encrypted indexing for spatial queries, and Dong et al. [22] addressed provider collusion through game-theoretic smart contracts. Despite improving auditability and robustness, these works often face scalability limitations or lack efficient structural binding when handling encrypted, replicated data across decentralized nodes.

B. Dynamic Auditing and Update Efficiency

Supporting efficient data dynamics (modification, insertion, deletion) is essential for practical storage. Existing approaches like Duan et al. [23] utilized succinct linked lists for index management but suffered from traversal overheads. Le et al. [24] optimized bandwidth for dynamic cold storage using coding techniques. Xu et al. [25] proposed BDACD, enabling ciphertext deduplication via T-Merkle trees. Yu et al. [26] introduced a double compressed auditing scheme to achieve storage efficiency. A notable recent advancement is Edasvic [27], which employs cryptographic accumulators to enable dynamic verification in industrial clouds. However, Edasvic necessitates the re-computation of witnesses for all peer blocks during data updates, resulting in linear computational overhead ($O(n)$) that limits scalability for large datasets. In contrast, our MPA leverages rank-based Merkle trees to achieve logarithmic update complexity ($O(\log n)$).

C. Polynomial Commitments and Aggregation

To enhance verification efficiency, polynomial commitment schemes (PCS) such as KZG [28] have been adopted to compress proofs. Wang et al. [29] proposed AAA-KSC, using structured polynomials for keyword-searchable encryption, highlighting the trend toward compact proofs. In the context of multi-copy auditing, BPCS [30] leverages polynomial commitments to achieve constant-size proofs. However, BPCS processes files independently and lacks global batching, causing verification costs to grow linearly with the number of files. Most recently, TLARDA [31] proposes a threshold label-aggregating scheme for decentralized environments. While it achieves concise proofs, the verification process incurs significant computational overhead due to the complex algebraic reconstruction of challenge parameters. Unlike these approaches, MPA integrates Merkle structural binding with homomorphic polynomial properties, supporting full cross-file batch verification while maintaining lightweight updates, effectively addressing the limitations of existing schemes.

III. Preliminaries

A. Bilinear Pairings

Let $\mathbb{G}$ and $\mathbb{G}_T$ be two cyclic multiplicative groups of prime order q, and let g be a generator of $\mathbb{G}$. A bilinear pairing is a

map:

$$\hat{e} : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T,$$

satisfying the following properties [32]:

- *Bilinearity:* For all $u, v \in \mathbb{G}$ and $a, b \in \mathbb{Z}_q^*$, it holds that:
$$\hat{e}(u^a, v^b) = \hat{e}(u, v)^{ab}.$$
- *Non-degeneracy:* $\hat{e}(g, g) \neq 1_{\mathbb{G}_T}$, i.e., the pairing does not map all input pairs to the identity in $\mathbb{G}_T$.
- *Computability:* There exists an efficient algorithm to compute $\hat{e}(u, v)$ for all $u, v \in \mathbb{G}$.

B. Polynomial Commitments

A polynomial commitment scheme allows a prover to commit to a polynomial $f(X)$ and later provide a succinct proof that it evaluates to a claimed value at any point $r \in \mathbb{Z}_q^*$, without revealing the polynomial itself.

We adopt the standard structured reference string (SRS)-based PCS proposed by Kate, Zaverucha, and Goldberg (KZG) [28]. Let the trusted setup generate a secret $\alpha \in \mathbb{Z}_q^*$ and publish the structured public key:

$$(g, g^{\alpha}, g^{\alpha^2}, \ldots, g^{\alpha^s}),$$

for some maximum polynomial degree s. Given a polynomial $f(X) = \sum_{k=0}^{d} c_k X^k$ with $d \leq s$, its commitment is computed as:

$$\mathsf{Com}(f) = g^{f(\alpha)} = \prod_{k=0}^{d} (g^{\alpha^k})^{c_k}.$$

To prove that $f(r) = v$ for some evaluation point r, the prover constructs a quotient polynomial:

$$h(X) = \frac{f(X) - v}{X - r},$$

and sends the proof $w = g^{h(\alpha)}$. The verifier checks:

$$\hat{e}\left(\frac{\mathsf{Com}(f)}{g^v},\ g\right) \stackrel{?}{=} \hat{e}(w,\ g^{\alpha - r}).$$

C. Merkle Tree Structures

A Merkle tree [33] is a binary hash tree used to compactly commit to a list of values while enabling efficient and verifiable membership proofs. In our scheme, the user constructs a Merkle tree over the set of tags $\{\sigma_{ij}\}$ generated for each block of a given file copy.

Each leaf node of the Merkle tree is defined as:

$$\mathsf{leaf}_{ij} = H(\sigma_{ij}),$$

where H is a collision-resistant cryptographic hash function. Internal nodes are recursively computed as:

$$\mathsf{Node} = H(\mathsf{LeftChild} \parallel \mathsf{RightChild}).$$

The final root Root_i represents a commitment to all tags $\{\sigma_{ij}\}$ associated with the ith copy of the file. This root is published on the blockchain.

During audit, the CSP must return the authentication path $\mathsf{MerklePath}_{ij}$ from the challenged leaf σ_{ij} to the root. The blockchain verifier recomputes the hash path and checks whether it matches the published root Root_i.

D. Blockchain-Based Randomness

To ensure audit unpredictability and prevent precomputed proof forgery, our scheme leverages blockchain-based randomness [34] to generate fresh challenge parameters for each audit. Specifically, we adopt a commit-and-reveal protocol[1] to produce a public, verifiable, and tamper-resistant random seed.

In each audit round, the blockchain executes the following process:

1) *Commit Phase:* A designated randomness contributor (e.g., the user or smart contract) submits a commitment $C = H(r_s \| \text{timestamp})$ to the chain, where r_s is a secret seed.
2) *Reveal Phase:* After a predefined delay, the contributor reveals r_s, and the blockchain verifies that C corresponds to $H(r_s \| \text{timestamp})$.
3) *Challenge Generation:* The revealed r_s is used to derive the challenge point r, the index set $\{a_j\}$, and the coefficient set $\{v_j\}$ through a pseudorandom function or hash-based expansion.

The unpredictability of the challenge relies on at least one participant being honest. To mitigate collusion, we adopt an open, incentive-based model where any rational blockchain node can contribute. Consequently, security rests on the economic assumption that bribing all independent contributors in a permissionless network is prohibitively expensive.

IV. Problem Formulation

A. System Model

We formalize the MPA auditing system into three components: the entities and their capabilities, the core algorithms involved, and the overall audit protocol flow. Fig. 2 illustrates the overall architecture of MPA.

1) Entities and Capabilities:

- *User:* The data owner who holds a file and wants to outsource it to decentralized storage while ensuring its long-term integrity. The user is responsible for key generation, preprocessing data, generating authentication metadata, and initiating contracts on the blockchain.
- *Cloud Service Providers:* A set of independent and potentially untrusted servers that store user data. Each CSP holds a unique encrypted copy of the original file and must respond to periodic audits.
- *Blockchain:* A decentralized smart contract platform that publishes audit parameters as well as the Merkle root of commitments, coordinates challenge generation, and verifies the responses returned by CSPs.

2) Algorithms:

- $\mathit{Setup}(1^{\lambda}) \to pp$: Run by a trusted authority to generate the public parameters pp based on security parameter λ.
- $\mathit{KeyGen}(pp) \to (sk, pk)$: Executed by the user. The user samples $\alpha \in \mathbb{Z}_q^*$ as the secret key sk and computes the structured public key $pk = (g, g^{\alpha}, \ldots, g^{\alpha^{s-1}})$.
- $\mathit{Storage}(F, sk) \to \mathsf{Copy}_i, \{\sigma_{ij}\}, \mathsf{Root}_i$: The user splits the file F into multiple blocks and encrypted copies Copy_i,

[1] https://github.com/randao/randao

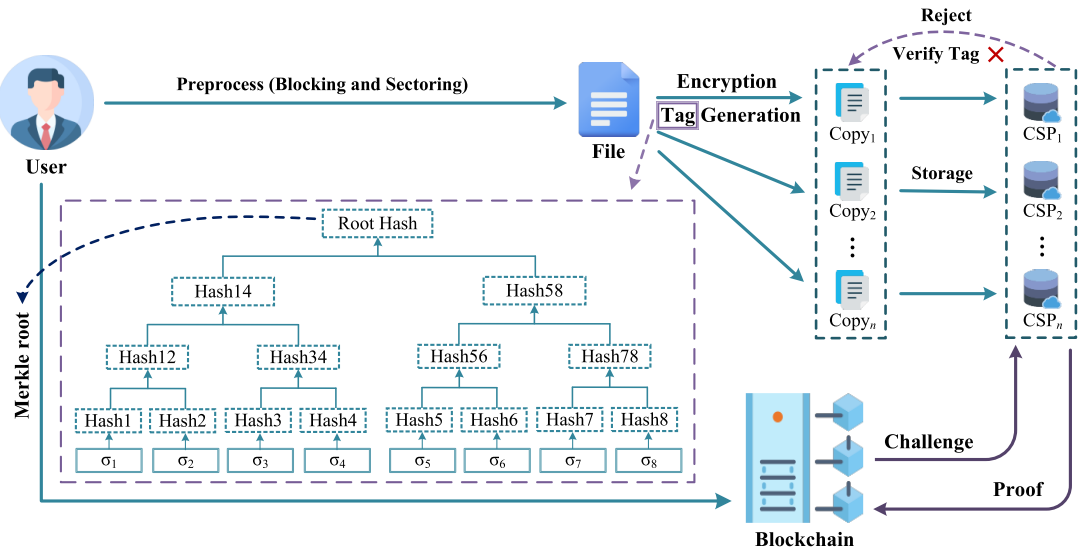


Fig. 2. System model of MPA auditing framework.

computes cryptographic tags σ_{ij} for each block using sk, organizes them into a Merkle tree, and uploads the root Root_i to the blockchain.

- *TagVerify*($\mathsf{Copy}_i, \mathsf{Root}_i$) $\rightarrow$ 0/1: Each CSP checks whether the copy Copy_i it receives correctly correspond to the Merkle root Root_i published on the blockchain. If verification fails, the CSP may reject the storage contract.
- *ChalGen*($\cdot$) $\rightarrow$ Chal: The blockchain generates a fresh challenge Chal consisting of a set of block indices $\{a_j\}$, a set of coefficients $\{v_j\}$, and a random evaluation point r.
- *Response*($\mathsf{Chal}, \mathsf{Copy}_i$) $\rightarrow$ $\mathcal{P}_i$: Each CSP computes a proof of possession $\mathcal{P}_i$ based on the challenged blocks, and their tags $\{\sigma_{ij}\}_{j\in\{a_j\}\in\mathsf{Chal}}$.
- *Verify*($\mathsf{Chal}, \mathcal{P}_i, \mathsf{Root}_i$) $\rightarrow$ 0/1: The blockchain verifies the integrity of the proof using pairing checks and Merkle path verification based on $\mathsf{Chal}, \mathcal{P}_i, \mathsf{Root}_i$.

3) Protocol Flow:

- *File Outsourcing* (Storage): The user splits a file into n blocks, each of which is further divided into s sectors. Each copy is uniquely encrypted and tagged using a polynomial commitment over the sectors. The user constructs a Merkle tree from the tags and stores the Merkle root Root_i on the blockchain as a public commitment.
- *Storage Verification* (TagVerify): Each CSP runs TagVerify to locally validate the correctness of received copy and the Merkle root. If the verification passes, the CSP accepts the storage task.
- *Challenge Generation* (ChalGen): At audit time, the blockchain executes ChalGen to generate and publish a random challenge $\mathsf{Chal} = (\{a_j, v_j\}_{j=1}^{c}, r)$ using a verifiable randomness process.
- *Proof Generation* (Response): Each CSP uses Response to evaluate:

$$\mathrm{val} = \sum_{j=1}^{c} v_j \cdot f_{m_{a_j}}(r),$$

and computes:

$$w = g^{h(\alpha)},$$

where

$$h(X) = \frac{f_M(X) - \mathrm{val}}{X - r},$$

along with the homomorphic authenticators and Merkle authentication paths.

- *Verification* (Verify): The blockchain executes Verify to check the pairing equation:

$$\hat{e}\left(\frac{\sigma^{\mathrm{agg}}}{g^{\mathrm{val}}}, g\right) \stackrel{?}{=} \hat{e}(w, g^{\alpha - r}),$$

where

$$\sigma^{\mathrm{agg}} = \prod_{j=1}^{c} \sigma_{a_j}^{v_j}, \sigma_{a_j} = g^{f_{m_{a_j}}(\alpha)},$$

and confirms the correctness of Merkle paths for each challenged tag.

B. Adversarial Model

We consider a semi-honest but potentially malicious environment where CSPs may attempt to mislead the blockchain during audits. The adversarial model focuses on the following assumptions and attack surfaces:

- *Malicious CSPs:* A malicious CSP may delete stored blocks to save costs while attempting to deceive the auditor. To pass audits, it may launch replay attacks by reusing previously valid proofs or attempt to forge polynomial commitments and Merkle authentication paths. Furthermore, multiple CSPs may collude to share storage resources and tags, thereby storing fewer copies than required.
- *Rational User:* Unlike the CSP, the user is assumed to be rational. Since they pay for data availability, they have no economic incentive to collude or disclose their secret key α to forge proofs, as this would compromise their

own data. Thus, security holds against a malicious CSP lacking α.

- *Adaptive Adversaries:* We consider adversaries that can choose which blocks to store or delete after seeing the system setup but before each audit. The MPA scheme is designed to remain secure against such adaptive strategies.

The adversary's goal is to produce a forged proof $(\sigma^*, w^*, \text{val}^*)$ such that:

$$\hat{e}\left(\frac{\sigma^*}{g^{\text{val}^*}},\ g\right) = \hat{e}(w^*, g^{\alpha - r}),$$

without storing the correct encrypted blocks corresponding to the challenged indices $\{a_j\}$. A successful forgery enables an incorrect file to pass verification, violating the audit integrity.

C. Security Model

We define the security of the proposed MPA scheme using a standard game-based model between a challenger $\mathcal{C}$ and a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$. The goal of $\mathcal{A}$ is to forge a valid proof of possession for some file blocks without actually storing the correct data.

1) Game Definition: The security game proceeds as follows:

- *Setup:* The challenger runs $\mathsf{Setup}(1^\lambda)$ and provides the public parameters pp to the adversary.
- *Key Generation:* The challenger runs $\mathsf{KeyGen}(pp) \rightarrow (sk, pk)$ and gives pk to the adversary, while keeping sk secret.
- *Storage Phase:* The challenger chooses a file F, splits it into n blocks and s sectors per block, and generates tags $\{\sigma_{ij}\}$ using $\mathsf{Storage}$. The root Root_i is published to the blockchain.
- *Query Phase:* The adversary may obtain any public data including $\{\sigma_{ij}\}$, Root_i, and previously issued challenge-response pairs. The adversary may adaptively delete blocks or simulate CSPs.
- *Challenge:* The challenger generates a random challenge $\mathsf{Chal} = (r, \{a_j\}, \{v_j\})$ using $\mathsf{ChalGen}$ and sends it to the adversary.
- *Forgery:* The adversary outputs a tuple $(\sigma^*, w^*, \text{val}^*)$ and Merkle paths Π^*, claiming to be a valid response.

2) Winning Condition: The adversary wins the game if:

1) The pairing check passes:

$$\hat{e}\left(\frac{\sigma^*}{g^{\text{val}^*}}, g\right) = \hat{e}(w^*, g^{\alpha - r}).$$

2) The Merkle paths Π^* are valid with respect to Root_i.
3) The challenged blocks $\{a_j\}$ include at least one block not correctly stored by the adversary.

3) Definition: We say the MPA scheme is (t, ε)-secure if for any t-time PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins the above game is at most ε. That is:

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Forge}}(\lambda) = \Pr[\mathcal{A}\text{wins}] \leq \varepsilon,$$

under the assumption that the PCS is binding and the hash functions are collision-resistant.

D. Security Goals

We now formalize the security guarantees that the MPA scheme aims to achieve in the presence of potentially malicious CSPs and public verifiers.

- *Correctness:* If all CSPs honestly follow the protocol and store the correct data blocks, then the verification on the blockchain will accept the proof. Formally, for any valid response generated over the challenged blocks, the pairing check and Merkle path verification will pass with overwhelming probability.
- *Unforgeability:* No PPT adversary that does not store the correct data can produce a valid proof $(\sigma^*, w^*, \mathsf{val}^*)$ such that:

$$\hat{e}\left(\frac{\sigma^*}{g^{\mathsf{val}^*}}, g\right) = \hat{e}(w^*, g^{\alpha - r}),$$

and corresponding Merkle paths verify, except with negligible probability. Furthermore, this requirement implies audit freshness. Since the verification equation is strictly bound to the fresh random challenge r, any attempt to replay outdated proofs from previous rounds will fail the pairing check.
- *Soundness (Update, Multi-Copy, and Multi-File):* The auditing scheme must guarantee that valid proofs bind strictly to the current data version, specific replica index, and file context. This ensures that malicious CSPs cannot pass audits by: (1) using stale data from previous versions, (2) colluding to store fewer copies, or (3) reusing blocks across different files.
- *Public Verifiability:* Anyone with access to the public parameters, the Merkle root, and the response proof can verify the integrity of the audited blocks without access to the user's secret key. This enables decentralized and transparent auditing on-chain.

E. Assumptions

The security of MPA relies on the following standard cryptographic assumptions:

- *t-Strong Diffie-Hellman (t-SDH) Assumption [28]:* Let $\mathbb{G}$ be a cyclic group of prime order q with generator g. Given a $(t+1)$-tuple $(g, g^\alpha, g^{\alpha^2}, \ldots, g^{\alpha^t})$ for a randomly chosen $\alpha \in \mathbb{Z}_q^*$, it is computationally infeasible for any PPT adversary to output a pair $\left(c, g^{\frac{1}{\alpha + c}}\right)$ where $c \in \mathbb{Z}_q \setminus \{-\alpha\}$.
- *Collision Resistance of Hash Functions:* The hash function $H(\cdot)$ used to construct the Merkle tree is assumed to be collision-resistant. Formally, it is computationally infeasible for an adversary to find two distinct inputs $x \neq y$ such that $H(x) = H(y)$.
- *Random Oracle Model:* We model the cryptographic hash functions used for challenge generation (including the blockchain-based commit-and-reveal process) as random oracles.

V. Proposed MPA

A. Overview

MPA integrates PCS with Merkle tree structures to achieve efficient, scalable auditing for decentralized storage. Data

TABLE I
NOTATIONS

Notation	Description
1^λ	Security parameter
q	A large prime number
$\mathbb{G}, \mathbb{G}_T$	Two cyclic multiplicative groups of order q
g	A generator of $\mathbb{G}$
$\hat{e}$	Bilinear pairing $\hat{e} : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$
$H(\cdot)$	Collision-resistant hash function
pp	Public parameter
(sk, pk)	User's secret and public key pair
s	Number of sectors per block
n	Number of blocks in file F
Copy_i	The ith encrypted copy of F
m_{ijk}	The kth sector of jth block in Copy_i
σ_{ij}	Commitment for jth block of Copy_i
Root_i	Merkle root of commitments in Copy_i
C_{ij}	Commitment recomputed by CSP_i
Chal	Audit challenge $\mathsf{Chal} = (\{a_j, v_j\}_{j=1}^c, r)$
c	The number of challenged blocks per file in each round
v_j	Random audit coefficient
$f_{m_{ij}}(X)$	Polynomial representation of jth block in Copy_i
val_i	Evaluation of aggregated polynomial at r
w_i	KZG proof for val_i
$\mathcal{P}_i$	Audit proof returned by CSP_i

blocks are encoded as polynomial commitments and organized into a Merkle tree, with the root serving as the on-chain anchor. For verification, MPA aggregates multiple block commitments into a succinct proof verified via a single pairing check, while Merkle paths ensure structural binding. Additionally, unique per-copy encryption prevents CSP collusion, and a blockchain-based commit-and-reveal protocol guarantees unbiased challenge generation.

B. Basic Scheme

The key notations used in the proposed scheme are summarized in Table I. The following subsections describe each algorithm component in detail.

Setup$(1^\lambda) \to pp$. A trusted center initializes system public parameters $pp = (\mathbb{G}, \mathbb{G}_T, q, g, \hat{e}, H)$.

Keygen$(pp) \to (sk, pk)$. The user selects a secret key $\alpha \in \mathbb{Z}_q^*$ and computes the public key:

$pk = (g, g^\alpha, g^{\alpha^2}, \ldots, g^{\alpha^{s-1}})$.

Storage$(F, sk) \to \mathsf{Copy}_i, \{\sigma_{ij}\}, \{\mathsf{Root}_i\}$. The original file F is divided into n blocks, each consisting of s sectors. The block structure is represented as:

$$F = \{b_{jk}\}_{1 \le j \le n,\ 1 \le k \le s}, \quad b_{jk} \in \mathbb{Z}_q.$$

The user generates N encrypted copies using symmetric encryption (e.g., AES). The ith copy is formed by:

$$m_{ijk} = \mathsf{Enc}_K(b_{jk} \| i),$$

where K is a symmetric encryption key and i is the copy index used as salt to ensure uniqueness. The encrypted file copy is then:

$$\mathsf{Copy}_i = \{m_{ijk}\}_{1 \le j \le n,\ 1 \le k \le s}.$$

For each encrypted block $m_{ij} = \{m_{ijk}\}_{k=1}^s$, the user constructs a degree-$(s-1)$ polynomial:

$$f_{m_{ij}}(X) = \sum_{k=1}^{s} m_{ijk} \cdot X^{k-1},$$

and computes its commitment using the secret key α:

$$\sigma_{ij} = g^{f_{m_{ij}}(\alpha)}.$$

All tags $\{\sigma_{ij}\}$ of Copy_i are hashed to form the leaf nodes of a Merkle Tree:

$$\mathsf{leaf}_{ij} = H(\sigma_{ij}).$$

The Merkle Tree is built in the standard bottom-up manner using the hash function H over child node pairs.

Finally, the Merkle root Root_i is uploaded to the blockchain, and the file copy Copy_i along with their Merkle authentication paths are sent to the ith storage server CSP_i.

TagVerify$(\mathsf{Copy}_i, \mathsf{Root}_i) \to 0/1$. CSP_i computes:

$$C_{ij} = \prod_{k=1}^{s} (g^{\alpha^{k-1}})^{m_{ijk}}.$$

Then reconstructs a Merkle tree using $\{C_{ij}\}$ as the leaf nodes (by applying $H(\cdot)$ to each commitment), and checks:

$$\mathsf{MerkleRoot}(\{C_{ij}\}) \stackrel{?}{=} \mathsf{Root}_i.$$

If the verification passes, the CSP_i retains $\{C_{ij}\}$, i.e., it retains $\{\sigma_{ij}\}$.

Algorithm 1 Blockchain-Based Challenge Generation

Input: File name $fname$, block count n, challenge count c, deposit amount d
Output: Challenge tuple $\mathsf{Chal} = (\{a_j, v_j\}_{j=1}^c, r)$
Step 1: User publishes an audit task $taskID$ with parameters $(fname, n, c, d)$
Step 2: Contract triggers multiple campaigns with commit-and-reveal phases
Step 3: Each participant submits hash commitment $H(\rho_i)$ along with a deposit
Step 4: After deadline, each participant reveals ρ_i and the contract checks consistency
Step 5: All valid ρ_i values are aggregated into a random seed rs via hashing
Step 6: Random indices $\{a_j\}_{j=1}^c \subseteq [1, n]$, weights $\{v_j\}_{j=1}^c \subseteq \mathbb{Z}_q^*$ and point r are derived from $rs \in \mathbb{Z}_q^*$
Step 7: The contract returns $\mathsf{Chal} = (\{a_j, v_j\}_{j=1}^c, r)$

ChalGen$(\cdot) \to \mathsf{Chal}$. The verifier selects a random subset of c block indices $\{a_j\}_{j=1}^c \subseteq \{1, \ldots, n\}$ uniformly, along with a random weights set $\{v_j\}_{j=1}^c \subseteq \mathbb{Z}_q^*$ and challenge point $r \in \mathbb{Z}_q^*$, as shown in Algorithm 1.

Response$(\mathsf{Chal}, \mathsf{Copy}_i) \to \mathcal{P}_i$. To support batch verification, CSP_i aggregates the selected c block polynomials into one using the set of random coefficients $\{v_j\}_{j=1}^c$. For each index a_j, the server reconstructs $f_{m_{i,a_j}}(X)$ and computes:

$$f_{M_i}(X) = \sum_{j=1}^{c} v_j \cdot f_{m_{i,a_j}}(X), val_i = f_{M_i}(r),$$

$$h_i(X) = \frac{f_{M_i}(X) - val_i}{X - r}, w_i = g^{h_i(\alpha)}.$$

Here, w_i acts as a polynomial evaluation proof at the point r under the PCS. Note that the proof element $w_i = g^{h_i(\alpha)}$ can be

computed by evaluating the polynomial $h_i(X)$ in the exponent using the structured public key. That is,

$$w_i = \prod_{k=0}^{\deg(h_i)} \left(g^{\alpha^k}\right)^{h_i[k]},$$

where $h_i[k]$ is the kth coefficient of $h_i(X)$.

Return proof:

$$\mathcal{P}_i = (val_i, w_i, \{\sigma_{i,a_j}, \mathsf{MerklePath}_{i,a_j}\}_{j=1}^{c}).$$

Verify$(\{\mathcal{P}_i\}_{i=1}^{N}, \{\mathsf{Root}_i\}, \mathsf{Chal}) \rightarrow 0/1$. Each proof $\mathcal{P}_i$ includes aggregation over c challenged blocks. The verifier computes:

$$\sigma^{\text{global}} = \prod_{i=1}^{N} \sigma_i^{\text{agg}} = \prod_{i=1}^{N}\prod_{j=1}^{c} \left(\sigma_{i,a_j}\right)^{v_j},$$

$$val^{\text{global}} = \sum_{i=1}^{N} val_i, w^{\text{global}} = \prod_{i=1}^{N} w_i.$$

Since all commitments are aggregated over c blocks per provider, the verifier performs a single global pairing check:

$$\hat{e}(\sigma^{\text{global}}/g^{val^{\text{global}}}, g) \stackrel{?}{=} \hat{e}(w^{\text{global}}, g^{\alpha - r}).$$

If the global check fails, fallback to per-provider checks:

$$\hat{e}(\sigma_i^{\text{agg}}/g^{val_i}, g) \stackrel{?}{=} \hat{e}(w_i, g^{\alpha - r}),$$

and for each challenged block $j = 1, \ldots, c$, verify that the corresponding σ_{i,a_j} is a valid leaf under the Merkle root Root_i using the provided path:

$$\mathsf{MerkleVerify}(\sigma_{i,a_j}, a_j, \mathsf{MerklePath}_{i,a_j}, \mathsf{Root}_i).$$

C. Extension to Multi-File and Multi-Copy Aggregated Verification

Assume there are T encrypted files, each denoted by $F^{(1)}, F^{(2)}, \ldots, F^{(T)}$. Each file $F^{(l)}$ is stored as N distinct encrypted copies across N CSPs, i.e., $\mathsf{Copy}_i^{(l)}$. For each file, the verifier selects a common random $\mathsf{Chal} = (\{a_j, v_j\}_{j=1}^{c}, r)$.

Each CSP_i generates a local aggregated proof for its corresponding copy of file $F^{(l)}$ as follows:

$$val_i^{(l)} = \sum_{j=1}^{c} v_j \cdot f_{m_{i,a_j}^{(l)}}(r),$$

$$w_i^{(l)} = g^{h_i^{(l)}(\alpha)}.$$

All proofs from all files and CSPs are then globally aggregated:

$$\sigma^{\text{global}} = \prod_{l=1}^{T}\prod_{i=1}^{N} \sigma_i^{(l)} = \prod_{l=1}^{T}\prod_{i=1}^{N}\prod_{j=1}^{c} \left(\sigma_{i,a_j}^{(l)}\right)^{v_j},$$

$$val^{\text{global}} = \sum_{l=1}^{T}\sum_{i=1}^{N} val_i^{(l)}, w^{\text{global}} = \prod_{l=1}^{T}\prod_{i=1}^{N} w_i^{(l)}.$$

The verifier performs a single pairing check:

$$\hat{e}(\sigma^{\text{global}}/g^{val^{\text{global}}}, g) \stackrel{?}{=} \hat{e}(w^{\text{global}}, g^{\alpha - r}).$$

In addition, for each of the c challenged blocks in each copy $\mathsf{Copy}_i^{(l)}$, the verifier checks the validity of the Merkle path against $\mathsf{Root}_i^{(l)}$:

$$\mathsf{MerkleVerify}\left(\sigma_{i,a_j}^{(l)}, a_j, \mathsf{MerklePath}_{i,a_j}^{(l)}, \mathsf{Root}_i^{(l)}\right).$$

D. Support for Dynamic Updates

A critical requirement for decentralized storage is the ability to efficiently support dynamic operations, including modification, insertion, and deletion. In traditional PDP schemes, authentication tags often bind the data block to its specific index (e.g., via $H(id\|j)$), causing a "re-indexing" bottleneck where inserting a block at index j shifts all subsequent indices, requiring $O(n)$ tag regenerations.

Our MPA scheme avoids the re-indexing penalty by design. The polynomial commitment tag $\sigma_{ij} = g^{f_{m_{ij}}(\alpha)}$ is *position-independent*, meaning it depends solely on the block content and the user's secret key, not on the block's physical index. Consequently, dynamic operations in MPA incur only logarithmic complexity $O(\log n)$.

We adopt a rank-based Merkle tree (or dynamic Merkle tree) structure, allowing blocks to be addressed by their logical rank. The specific procedures and their complexities are as follows:

- *Modification*: To modify the j-th block m_{ij} to m'_{ij}, the user computes the new tag σ'_{ij} (1 exponentiation), updates the leaf to $H(\sigma'_{ij})$, and recomputes the $\lceil \log_2 n \rceil$ hashes along the path to the root.
- *Insertion*: To insert a new block m^* at index j, the user computes its tag σ^* (1 exponentiation), inserts a new leaf $H(\sigma^*)$ at position j, and updates the affected path ($\lceil \log_2 n \rceil$ hashes). This increases the tree size by 1. Subsequent tags remain valid without re-computation.
- *Deletion*: To delete the block at index j, the corresponding leaf is removed, and the path to the root is updated ($\lceil \log_2 n \rceil$ hashes), reducing the tree size by 1.

For any dynamic operation, the cost is strictly limited to one group exponentiation (for new tags), $O(\log n)$ lightweight hash operations for path updates, and refreshing the Merkle Root. No other global parameters or accumulators need to be updated. This efficient handling aligns with practical requirements for decentralized storage.

E. Remark on PDP Vs. PoR

We explicitly clarify that MPA is designed as a PDP scheme. Its security goal is to detect data modification or deletion with high probability, ensuring ciphertext integrity. While PoR provides a stronger guarantee that the file can be fully extracted, this typically requires combining integrity checks with erasure coding (e.g., Reed-Solomon codes) and an extractor argument. MPA is compatible with such a PoR layering: a user can pre-encode the file using an (n, k) erasure code before generating the polynomial commitments and Merkle tree. In this setting, MPA audits ensure that the number of corrupted blocks does not exceed the erasure correction threshold $n - k$, thereby guaranteeing retrievability. However, the implementation of erasure coding is orthogonal to the core auditing logic and is considered an extensibility feature.

VI. Security Analysis

In this section, we analyze the security properties of the proposed MPA scheme. Specifically, we show that the scheme satisfies the security goals defined in Section IV-D under the assumptions outlined in Section IV-E.

A. Correctness

If all CSPs store the correct encrypted copies of the file and execute the protocol honestly, then the audit proof generated during the Response phase will be accepted by the blockchain verifier.

Let the aggregated polynomial over c challenged blocks be:

$$f_M(X) = \sum_{j=1}^{c} v_j \cdot f_{m_{a_j}}(X).$$

Then the evaluation at the challenge point r is:

$$\mathsf{val} = f_M(r),$$

and the quotient polynomial is:

$$h(X) = \frac{f_M(X) - \mathsf{val}}{X - r}.$$

The KZG proof is computed as:

$$w = g^{h(\alpha)},$$

and the commitment:

$$\sigma^{\mathsf{agg}} = g^{f_M(\alpha)}.$$

Substituting into the pairing verification:

$$\hat{e}\left(\frac{\sigma^{\mathsf{agg}}}{g^{\mathsf{val}}}, g\right) = \hat{e}(g^{h(\alpha)}, g^{\alpha - r}) = e(w, g^{\alpha - r}),$$

which holds by the correctness of polynomial commitment. The Merkle paths are also verifiable since each σ_{a_j} is correctly placed in the tree. Hence, honest behavior leads to audit acceptance. Crucially, since the verification equation relies solely on public parameters (i.e., $\sigma^{agg}, val, w, Root$) and requires no secret key, the proposed scheme achieves public verifiability 1, allowing any third party (e.g., blockchain nodes) to execute the audit transparently.

B. Unforgeability

We demonstrate that any PPT adversary $\mathcal{A}$ attempting to generate a valid audit proof without storing the correct data blocks cannot succeed except with negligible probability.

Theorem 1: The MPA scheme is unforgeable if the t-Strong Diffie-Hellman (t-SDH) [28] assumption holds in the bilinear group $\mathbb{G}$.

Proof: Suppose an adversary $\mathcal{A}$ outputs a forged tuple $(\sigma^*, \mathsf{val}^*, w^*)$ that satisfies the verification equation at a challenge point r:

$$\hat{e}\left(\sigma^* \cdot g^{-\mathsf{val}^*}, g\right) = \hat{e}(w^*, g^{\alpha - r}) \tag{1}$$

where $\mathsf{val}^* \neq f_M(r)$. Let $f_M(X)$ be the honest polynomial commitment and $h(X) = \frac{f_M(X) - f_M(r)}{X - r}$ be the correct quotient. The forged witness w^* must satisfy the relation in the exponent:

$$\log_g(w^*) = \frac{f^*(\alpha) - \mathsf{val}^*}{\alpha - r} \tag{2}$$

If $\mathcal{A}$ successfully forges the proof, we can construct an algorithm $\mathcal{B}$ that solves the t-SDH problem. Given the t-SDH challenge $(g, g^{\alpha}, \ldots, g^{\alpha^t})$, $\mathcal{B}$ sets up the system parameters and provides them to $\mathcal{A}$. When $\mathcal{A}$ returns the forged $(\sigma^*, \mathsf{val}^*, w^*)$, $\mathcal{B}$ performs the following polynomial long division:

$$f_M(X) - \mathsf{val}^* = h^*(X)(X - r) + \gamma \tag{3}$$

where $\gamma = f_M(r) - \mathsf{val}^*$ is a non-zero constant since $\mathsf{val}^* \neq f_M(r)$. In the exponent, this relation is:

$$g^{f_M(\alpha) - \mathsf{val}^*} = (g^{\alpha - r})^{h^*(\alpha)} \cdot g^{\gamma} \tag{4}$$

By rearranging the verification equation (1), we have $w^* = g^{(f_M(\alpha) - \mathsf{val}^*)/(\alpha - r)}$. Substituting this into the division result:

$$w^* = g^{h^*(\alpha) + \frac{\gamma}{\alpha - r}} \implies \left(\frac{w^*}{g^{h^*(\alpha)}}\right)^{1/\gamma} = g^{\frac{1}{\alpha - r}} \tag{5}$$

Since $h^*(X)$ is a known polynomial of degree at most $t - 1$, $\mathcal{B}$ can compute $g^{h^*(\alpha)}$ using the challenge parameters. Thus, $\mathcal{B}$ extracts the t-SDH solution $g^{1/(\alpha - r)}$. The success of the forgery implies:

- Collision Resistance: $\mathcal{A}$ found $f^*(X) \neq f_M(X)$ such that $g^{f^*(\alpha)} = g^{f_M(\alpha)}$, which breaks the binding property of KZG commitments.
- t-SDH Hardness: If the commitments are binding, $\mathcal{A}$ must have constructed a valid witness for a false evaluation, which directly provides a solution to the t-SDH problem as shown in Eq. (5).

Since t-SDH is computationally hard, the probability of $\mathcal{A}$'s success is negligible.

C. Security Analysis of Merkle-Based Authentication

Addressing the soundness requirements defined in Section IV-D, we demonstrate how the Merkle tree enforces strict structural binding to satisfy update soundness and multi-copy/multi-file soundness. This analysis highlights the critical role of the Merkle structure in preventing CSPs from forging valid proofs using outdated, misplaced, or mismatched data blocks. Let $\mathcal{H} : \{0,1\}^* \to \{0,1\}^{\lambda}$ be a collision-resistant hash function. The formal analysis proceeds as follows:

1) *Structural Binding.* We provide a formal reduction to the collision resistance of the hash function $\mathcal{H}$. Let Π_j be the valid Merkle authentication path for a tag σ_{ij} at logical index j. The verification process is deterministic, defined by a reconstruction function $\mathcal{R}$ such that $\mathcal{R}(\sigma_{ij}, j, \Pi_j) = Root_i$.

   Theorem 2: For any probabilistic PPT adversary $\mathcal{A}$, the probability of successfully forging a structural inconsistency is negligible, bounded by the advantage of breaking the collision resistance of $\mathcal{H}$.

   Proof: Suppose $\mathcal{A}$ outputs a forged tuple (σ^*, j^*, Π^*) that passes verification with respect to $Root_i$, where $(\sigma^*, j^*) \neq (\sigma_{ij}, j)$ (i.e., either the tag value or the position is falsified). Since the proof is valid, it holds that:

$$\mathcal{R}(\sigma^*, j^*, \Pi^*) = Root_i$$

   Given that the honest path also satisfies $\mathcal{R}(\sigma_{ij}, j, \Pi_j) = Root_i$, and the inputs at the leaf level differ, there must

exist a specific height h in the Merkle tree where the hash inputs differ ($x_h \neq y_h$) but their hash outputs are identical ($\mathcal{H}(x_h) = \mathcal{H}(y_h)$). This constitutes a collision for $\mathcal{H}$. Therefore, the advantage of the adversary $\mathcal{A}$ in breaking the structural binding is bounded by the advantage of finding a collision in $\mathcal{H}$:

$$Adv_{\mathcal{A}}^{Struct}(\lambda) \leq Adv_{\mathcal{H}}^{Coll}(\lambda)$$

Since $\mathcal{H}$ is collision-resistant, $Adv_{\mathcal{H}}^{Coll}(\lambda)$ is negligible, and thus $Adv_{\mathcal{A}}^{Struct}(\lambda)$ is negligible.

2): *Update Soundness.* Let S_t denote the storage state at time t with Merkle root $Root_t$. A dynamic update operation (e.g., $Update(m_{ij} \rightarrow m'_{ij})$) transitions the state $S_t \rightarrow S_{t+1}$, resulting in a new root:

$$Root_{t+1} \neq Root_t$$

If a malicious CSP attempts to satisfy an audit at time $t+1$ using a stale proof π_t generated from state S_t (where π_t is valid w.r.t $Root_t$), the verification check fails:

$$Verify(Root_{t+1}, \pi_t) = 0$$

This holds because verifying π_t reconstructs $Root_t$, and $Root_t \neq Root_{t+1}$. Thus, replay attacks are strictly prevented.

3) *Multi-Copy Soundness.* To prevent cross-copy collusion, the user encrypts the j-th block of the i-th copy ($i \in [1, N]$) using the copy index i as a unique salt. The encrypted sectors m_{ijk} are derived as:

$$m_{ijk} = Enc_K(b_{jk} \| i)$$

This dependency on i generates distinct polynomial coefficients, resulting in distinct tags $\sigma_{ij} = g^{f_{m_{ij}}(\alpha)}$ and distinct leaf nodes $L_{ij} = \mathcal{H}(\sigma_{ij})$ for each copy. Consequently, two security guarantees hold: (1) The aggregated Merkle Roots for any two copies are distinct ($Root_i \neq Root_{i'}$). Thus, a proof borrowed from Copy i fails the Merkle reconstruction check for Copy i': $\mathcal{R}(\sigma_{ij}, \Pi) \neq Root_{i'}$. (2) Recomputing a valid tag $\sigma_{i'j}$ for a missing copy requires knowledge of the unique polynomial coefficients corresponding to that specific encrypted replica. Since the replicas are uniquely differentiated and the colluding CSP does not possess the actual data of the missing copy, it cannot construct the correct polynomial to derive the valid tag using the public parameters. Furthermore, extending to the multi-file setting, the unique Merkle root $Root_i^{(l)}$ acts as a cryptographically secure domain separator. Even if an adversary observes identical plaintext blocks across different files $F^{(A)}$ and $F^{(B)}$, the inclusion of file-specific metadata in the Merkle tree construction ensures that the authentication paths are distinct. Consequently, a valid proof path Π_A for a block in File A cannot be successfully verified against the root of File B ($\mathcal{R}(\sigma, \Pi_A) \neq Root_B$), thereby mathematically precluding cross-file copy-and-paste attacks.

D. Security of Blockchain Randomness

We analyze the security of the commit-and-reveal protocol against last-revealer bias and liveness attacks, taking into account the chosen parameters and slashing policy.

1) Liveness Analysis: To prevent "non-reveal griefing", our aggregation logic (Algorithm 1, Step 5) is non-blocking. The final random seed is derived as $r_s = H(\{\rho_i\}_{i \in \text{Valid}})$. If a participant fails to reveal within the specified window, they are penalized, and the protocol proceeds with the remaining valid secrets. This ensures the liveness of the audit process regardless of participant behavior.

2) Quantitative Analysis of Bias: It is well known that the last revealer can influence the randomness by choosing whether to reveal or withhold their secret [34]. However, in our auditing scenario, we mitigate this via economic deterrence. The adversary's goal is to bias the challenge indices $\{a_j\}$ to exclude corrupted blocks. Let d be the deposit amount and V_{saved} be the storage cost saved by deleting data. The adversary can influence the outcome between two seeds: S_{reveal} and $S_{withhold}$. The attack is economically rational only if the expected gain from evasion exceeds the slashing penalty:

$$V_{saved} \times (P_{escape}(S_{withhold}) - P_{escape}(S_{reveal})) > d$$

Given that the detection probability for $c = 460$ is 99%, the baseline escape probability $P_{escape} \approx 1\%$. The marginal gain from selecting the better of two random seeds is negligible compared to a sufficiently large deposit d. Thus, we quantify the security by requiring $d > V_{saved}$, which renders the bias attack irrational.

3) Comparison With Alternatives: While alternatives like verifiable random functions (VRFs) or beacon chain randomness offer stronger unbiasedness, they introduce external oracle dependencies or specific chain constraints. Our commit-and-reveal approach is fully decentralized, chain-agnostic, and minimizes operational costs, making it the optimal choice for our target lightweight deployment environment.

E. Probabilistic Detection Guarantee

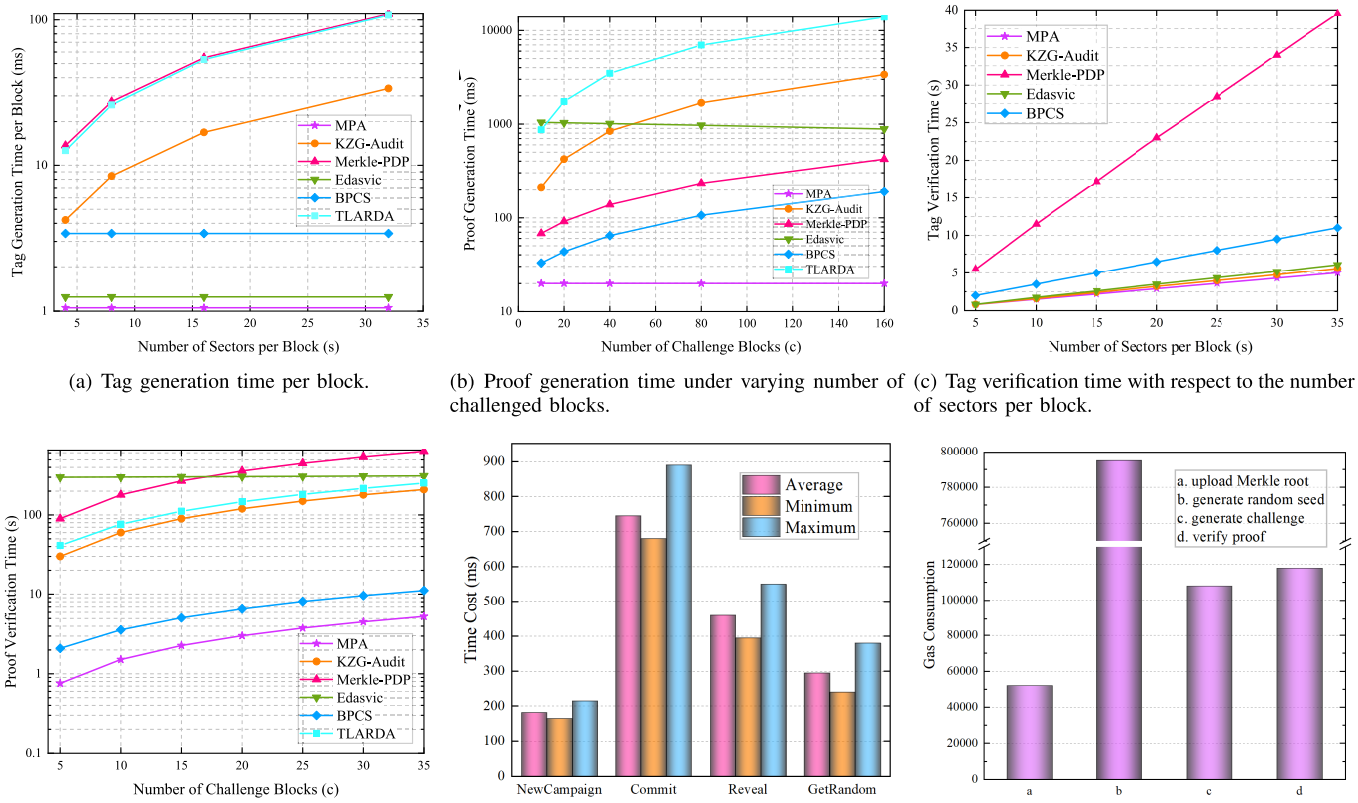
Since MPA employs probabilistic sampling (random challenge $\{a_j\}$), we quantify the detection probability relative to the corruption rate. Let $\alpha \in [0, 1]$ denote the fraction of corrupted blocks maintained by a malicious CSP. The adversary passes the audit only if none of the c challenged blocks fall into the corrupted set. Assuming the challenge indices are chosen uniformly at random, the probability of detecting at least one corrupted block is:

$$P_{detect} = 1 - (1 - \alpha)^c$$

Thus, the soundness error (probability of false acceptance) is $\epsilon = (1 - \alpha)^c$. This relationship provides clear guidance for parameter selection. To achieve a target detection probability P_{target} against a minimum corruption rate α_{min}, the challenge size c must satisfy:

$$c \geq \frac{\ln(1 - P_{target})}{\ln(1 - \alpha_{min})}$$

For example, to detect data corruption with 99% probability ($P_{target} = 0.99$) when at least 1% of the file is corrupted ($\alpha_{min} = 0.01$), the verifier must set $c \approx 460$. This constant challenge size ensures high security assurance independent of the total file size n, maintaining the lightweight nature of the protocol.



(a) Tag generation time per block.

(b) Proof generation time under varying number of challenged blocks.

(c) Tag verification time with respect to the number of sectors per block.

(d) Proof verification time under varying number of challenged blocks.

(e) Time cost of each function in RANDAO.

(f) Gas consumption of each operation.

Fig. 3. The actual performance analysis of off-chain and on-chain costs.

TABLE II
THEORETICAL COMPARISON OF AUDITING SCHEMES

Scheme	Proof Size	Pairing Complexity	Update	Batching	Binding
MPA	$O(c\log_2 n)$	$O(1)$	Yes	Full	Structural
KZG-Audit	$O(c)$	$O(c)$	No	Partial	Weak
Merkle-PDP	$O(c\log_2 n)$	$O(c)$	No	No	Moderate
Edasvic	$O(c)$	$O(1)$	Yes	Partial	Index-based
BPCS	$O(1)$	$O(1)$	No	Partial	Index-based
TLARDA	$O(1)$	$O(1)$	No	Partial	Index-based

TABLE III
NOTATIONS AND COMPUTATIONAL COSTS OF CRYPTOGRAPHIC OPERATIONS

Symbol	Operation	Cost (ms)
$\mathcal{H}_{\mathbb{G}}$	Hashing into $\mathbb{G}$	1.295
$\mathcal{H}$	Standard hash (SHA-256)	0.001
P	Bilinear pairing operation	0.951
$\mathsf{Exp}_{\mathbb{G}}$	Exponentiation in $\mathbb{G}$	1.054
$\mathsf{Exp}_{\mathbb{G}_T}$	Exponentiation in $\mathbb{G}_T$	0.182

VII. PERFORMANCE EVALUATION

We evaluate MPA against five representative schemes: KZG-Audit [28], Merkle-PDP [10], Edasvic [27], BPCS [30], and TLARDA [31]. These baselines encompass diverse auditing paradigms including hash-based, commitment-based, and accumulator-based approaches. Specifically, Merkle-PDP and KZG-Audit provide foundational hash and commitment structures. Edasvic focuses on accumulator-based auditing while BPCS and TLARDA serve as representative schemes utilizing polynomial-based algebraic structures for auditing optimization. Our evaluation focuses on auditing efficiency, verification scalability, and update performance. Theoretical comparisons are summarized in Table II, and experimental results are reported in Fig. 3.

A. Theoretical Analysis

Our analysis focuses on three key metrics: computation cost, communication overhead, and storage complexity. To quantify computation costs, we summarize the major cryptographic operations in Table III. For storage and communication analysis, we denote element sizes in $\mathbb{Z}_p$, $\mathbb{G}$, and $\mathbb{G}_T$ as $|\mathbb{Z}_p|$, $|\mathbb{G}|$, and $|\mathbb{G}_T|$, respectively. The cost metrics consider the number of files T, number of replicas N, number of blocks per file n, number of sectors per block s, and number of challenged blocks c. The summarized results are presented in Tables IV, V, and VI.

Table IV provides a comprehensive comparison of computation costs among the evaluated auditing schemes. For tag generation, MPA achieves significant efficiency gains by minimizing cryptographic operations at the block level. Specifically, while foundational schemes like KZG-Audit and Merkle-PDP require group exponentiations for every sector, MPA leverages sector-level polynomial aggregation to reduce the overhead to approximately one group exponentiation per block. Although Edasvic, BPCS and TLARDA also leverage polynomial or accumulator structures to optimize tag costs, MPA's integration with a Merkle-based framework ensures that this efficiency does not compromise structural integrity

TABLE IV
COMPUTATION COST COMPARISON

Scheme	Tag Gen	Proof Gen	Tag Verify	Proof Verify
KZG-Audit	$TNns \cdot \mathrm{Exp}_{\mathbb{G}}$	$TNcs \cdot \mathrm{Exp}_{\mathbb{G}}$	$TNns \cdot \mathrm{Exp}_{\mathbb{G}}$	$2TNcs \cdot P$
Merkle-PDP	$TN(ns(2\mathrm{Exp}_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}}) + 2n\mathcal{H})$	$TN((c+3)\mathrm{Exp}_{\mathbb{G}} + (c+1)\mathcal{H}_{\mathbb{G}} + c\log_2 n \cdot \mathcal{H})$	$2TNns(2\mathrm{Exp}_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}}) + 2TNn \cdot \mathcal{H}$	$2TNcs(2\mathrm{Exp}_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}}) + TNc(\log_2 n + 1) \cdot \mathcal{H}$
Edasvic	$TN(n+1)\mathrm{Exp}_{\mathbb{G}} + TNn\mathcal{H}$	$TN(n-c+2)\mathrm{Exp}_{\mathbb{G}}$	$TN(ns+1)\mathrm{Exp}_{\mathbb{G}}$	$TN(2P + (2n+c+2)\mathrm{Exp}_{\mathbb{G}} + c\mathcal{H})$
BPCS	$TNn(2\mathrm{Exp}_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}})$	$TN((c+s+1)\mathrm{Exp}_{\mathbb{G}} + \mathcal{H})$	$TNn(2P + s \cdot \mathrm{Exp}_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}})$	$T(4P + N(c(\mathrm{Exp}_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}}) + 2\mathrm{Exp}_{\mathbb{G}} + \mathcal{H}) + 2\mathrm{Exp}_{\mathbb{G}})$
TLARDA	$TNns(2Exp_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}})$	$TN \cdot c \cdot s(P + 2Exp_{\mathbb{G}} + \mathcal{H}_{\mathbb{G}})$	N/A	$TN(P + (c+1)Exp_{\mathbb{G}} + c\mathcal{H}_{\mathbb{G}})$
MPA (ours)	$TNn(\mathrm{Exp}_{\mathbb{G}} + 2\mathcal{H})$	$TN((s-1)\mathrm{Exp}_{\mathbb{G}} + c\log_2 n \cdot \mathcal{H})$	$TNns \cdot \mathrm{Exp}_{\mathbb{G}} + 2TNn \cdot \mathcal{H}$	$2P + (TNc+2)\mathrm{Exp}_{\mathbb{G}} + TNc(\log_2 n + 1) \cdot \mathcal{H}$

Notation: T = number of files, N = number of replicas, n = number of data blocks per file, s = number of sectors per block, c = number of challenged blocks. $\mathcal{H}$ denotes standard hash operations (e.g., SHA-256 or Map-to-Scalar), while $\mathcal{H}_{\mathbb{G}}$ denotes map-to-group hash operations.

TABLE V
AUDIT-TIME COMMUNICATION OVERHEAD (PROOF SIZE)

Scheme	Proof Size
KZG-Audit	$(c+1)\lvert\mathbb{G}\rvert + \lvert\mathbb{Z}_p\rvert$
Merkle-PDP	$c\lvert\mathbb{G}\rvert + (c\log_2 n + c + 1)\lvert\mathbb{Z}_p\rvert$
Edasvic	$(c+2)\lvert\mathbb{G}\rvert + 2\lvert\mathbb{Z}_p\rvert$
BPCS	$3\lvert\mathbb{G}\rvert + \lvert\mathbb{Z}_p\rvert$
TLARDA	$\lvert\mathbb{G}_T\rvert + \lvert\mathbb{Z}_p\rvert$
MPA (ours)	$(c+1)\lvert\mathbb{G}\rvert + (c\log_2 n + 1)\lvert\mathbb{Z}_p\rvert$

Note: The output size of a hash function (e.g., a Merkle path computed via SHA-256) is typically equal to $|\mathbb{Z}_p|$, so we use $|\mathbb{Z}_p|$ to represent the size of hash values in our analysis.

TABLE VI
STORAGE OVERHEAD COMPARISON (ON-CHAIN AND OFF-CHAIN)

Scheme	Off-chain (CSP)	On-chain (Blockchain)
KZG-Audit	$TNns\lvert\mathbb{G}\rvert$	–
Merkle-PDP	$TNns(\lvert\mathbb{G}\rvert + (\log_2 n + 2)\lvert\mathbb{Z}_p\rvert)$	–
Edasvic	$TN(n+1)\lvert\mathbb{G}\rvert$	–
BPCS	$TNn\lvert\mathbb{G}\rvert$	$3TN\lvert\mathbb{G}\rvert + (TN + 2c)\lvert\mathbb{Z}_p\rvert$
TLARDA	$TNn\lvert\mathbb{G}\rvert$	–
MPA (ours)	$TNn(\lvert\mathbb{G}\rvert + (\log_2 n + 1)\lvert\mathbb{Z}_p\rvert)$	$TN(\lvert\mathbb{Z}_p\rvert)$

or update flexibility. MPA maintains optimal proof generation efficiency, requiring $TN(s-1)$ exponentiations. Notably, the cost is dominated by the polynomial degree s, while the number of challenged blocks c only incurs lightweight scalar hashing. Tag and proof verification cost is where MPA demonstrates its scalability, offering significant advantages over other schemes. Overall, the analysis shows that MPA achieves the most balanced tradeoff between computation, scalability, and structural integrity, particularly in high-redundancy or multi-file environments. Regarding dynamic updates, as detailed in Section V-D, MPA incurs a logarithmic computation overhead $O(\log n)$ for modification, insertion, and deletion due to the position-independent nature of polynomial commitments and the rank-based Merkle structure. In contrast, traditional schemes often require linear $O(n)$ re-indexing or accumulator re-computation.

We analyze the communication overhead incurred during the proof generation, as shown in Table V. Our MPA scheme incurs $(c+1)|\mathbb{G}|$ due to the inclusion of tags and witness, and $(c\log_2 n+1)|\mathbb{Z}_p|$ from the Merkle authentication paths and evaluation scalars, where the depth of the Merkle tree determines that the size of the Merkle path is equal to $(\log_2 n+1)|\mathbb{Z}_p|$. The communication complexity derived above assumes independent authentication paths for each challenged block, resulting in a conservative upper bound of $O(c\log_2 n)$. We note that standard compression techniques, such as Merkle multiproofs, can be applied to our protocol to eliminate redundant sibling nodes in the authentication paths. This would strictly reduce the communication overhead, especially when the challenged blocks are clustered, without altering the underlying security guarantees. For simplicity and to provide a worst-case baseline, we maintain the analysis based on standard Merkle paths. Compared to others, MPA achieves a well-balanced proof size without sacrificing too much efficiency, supports structure binding and multi-replica auditing with moderate bandwidth cost.

Discussion on Trade-offs with Vector Commitments: It is worth noting that succinct vector commitments (e.g., KZG) could theoretically reduce the membership proof size to constant scale ($O(1)$). However, we adopt the Merkle-based approach to achieve an optimal balance for lightweight decentralized auditing. First, vector commitments typically incur significantly higher computational costs during proof generation and updates (requiring $O(n)$ group exponentiations versus $O(n)$ lightweight hashes) and often rely on a trusted setup. Second, a core requirement of MPA is efficient dynamic updating. Merkle trees allow updates with $O(\log n)$ hash operations, whereas updating vector commitments and maintaining valid witnesses for all blocks typically involves expensive algebraic operations. Thus, we accept the logarithmic communication cost of Merkle paths to ensure setup-free security and low-latency updates.

Table VI compares the storage overhead of each scheme, distinguishing between off-chain and on-chain storage. On-chain storage includes public commitments such as root hashes and audit logs. KZG-Audit incurs off-chain storage of $TNns|\mathbb{G}|$

due to the need for storing per-sector tags. Merkle-PDP stores tags and Merkle authentication paths off-chain, leading to an overhead of $TNn(|\mathbb{G}| + (\log_2 n + 2)|\mathbb{Z}_p|)$. The off-chain cost $TN(n+1)|\mathbb{G}|$ of Edasvic consists of block tags and a block accumulator for each file. Both TLARDA and BPCS optimize space by maintaining an off-chain tag storage of $TNn|\mathbb{G}|$, corresponding to one algebraic signature per block. However, BPCS additionally incurs a significant on-chain audit log cost of $3TN|\mathbb{G}| + (TN + 2c)|\mathbb{Z}_p|$. In contrast, our MPA scheme strikes a balance by requiring $TNn(|\mathbb{G}| + (\log_2 n + 1)|\mathbb{Z}_p|)$ off-chain, accounting for tags and Merkle paths, while keeping on-chain storage minimal with only $TN|\mathbb{Z}_p|$, which stores Merkle roots. This separation ensures strong verifiability with practical deployment overhead.

In summary, MPA achieves optimal asymptotic performance across key metrics. It minimizes computation overhead by leveraging algebraic aggregation and structural Merkle authentication, offering a lightweight and secure solution for decentralized multi-copy auditing.

B. Experimental Evaluation

1) Off-Chain Costs: MPA is implemented in Python using the Charm-Crypto library with a 256-bit prime-order Type-III pairing group. AES uses a 128-bit key. All experiments are conducted on an Intel Core i7 CPU with 16 GB RAM. By default, we fix the number of blocks $n = 1000$, sectors per block $s = 20$, and challenged blocks per audit $c = 20$. Our evaluation focuses on three key aspects: tag and proof generation time, and verification time.

We evaluate the tag generation time for different schemes under varying numbers of sectors per block. As shown in Fig. 3(a), our MPA scheme consistently exhibits the lowest cost, requiring only one exponentiation per block regardless of the sector count. KZG-Audit, Merkle-PDP and TLARDA incur a cost linear in the number of sectors, as they perform an exponentiation for each sector. BPCS and Edasvic remain constant with respect to s, but their tag generation time is notably higher than MPA due to either double exponentiations or additional accumulators. These findings demonstrate that MPA offers the best trade-off between expressiveness and efficiency.

Fig. 3(b) shows the proof generation time with varying numbers of challenged blocks c, where we fix $n = 1000$ and $s = 20$. MPA maintains a stable cost of $(s-1)\text{Exp}_{\mathbb{G}}$, resulting in the lowest and constant runtime across all values of c. In contrast, KZG-Audit incurs linear growth in $c \cdot s$, and BPCS requires $(c+s+1)\text{Exp}_{\mathbb{G}}$, both scaling linearly with c. TLARDA incurs a linear computational cost necessitated by the aggregation of algebraic signatures for each challenged block. Edasvic's cost decreases with c, but remains higher due to accumulator structure. Merkle-PDP exhibits high proof generation time as it involves both multiple exponentiations and costly hash operations, with complexity $(c+3)\text{Exp}_{\mathbb{G}} + (c+1)\mathcal{H}_{\mathbb{G}}$.

Fig. 3(c) presents the tag verification time as the number of sectors per block increases, where we fix $T = 50, N = 3$ and $n = 1000$. All schemes exhibit linear growth since each sector requires a separate tag or hash computation. Among them, Merkle-PDP incurs the highest cost due to multiple group exponentiations and hash operations per sector. BPCS also shows noticeable overhead due to its pairing and hash operation. In contrast, our MPA scheme maintains the lowest verification cost, outperforming all other schemes by leveraging efficient per-sector exponentiation without additional hashing. Edasvic and KZG-Audit follow closely but remain costlier than MPA.

Fig. 3(d) shows the variation of proof verification time as the number of challenged blocks increases, where we fix $T = 50, N = 3, n = 1000$ and $s = 20$. The proposed MPA scheme consistently outperforms other approaches, maintaining sub-linear growth due to its aggregate verification mechanism. In contrast, KZG-Audit and TLARDA grow linearly with the challenge size. Specifically, KZG-Audit requires a pairing check for each proof, while TLARDA incurs computational overhead for algebraic signature reconstruction for each challenged block. Merkle-PDP incurs significant overhead from verifying inclusion paths and checking signatures, resulting in the highest cost. Edasvic is almost unchanged, since c is significantly smaller than n. BPCS exhibits moderate growth, as it performs pairing operations per file proof. Overall, MPA achieves the best balance of efficiency and scalability.

Note that our evaluation in Fig. 3(d) explicitly accounts for the computational overhead of Merkle path verification (using SHA-256). Although the number of required hash operations scales with $O(c \log n)$, the actual runtime impact is marginal. This is because a standard cryptographic hash operation (approx. 0.001 ms) is orders of magnitude faster than a bilinear pairing operation ($P \approx 0.951$ ms) or a group exponentiation ($Exp_{\mathbb{G}} \approx 1.054$ ms) as listed in Table III. Consequently, even with structural verification included, MPA maintains a significant efficiency advantage over baseline schemes like KZG-Audit and Merkle-PDP, which rely on computationally expensive operations that scale linearly with the challenge size. MPA enables efficient block-level updates, requiring only localized Merkle path recomputation and a single tag update. Edasvic requires re-accumulating all tags, leading to over 200 ms cost. Other schemes lack update support.

MPA demonstrates clear advantages across all experimental metrics. It achieves low computation cost, localized updates, and scalable auditing. Compared to prior works, MPA balances structural binding, efficiency, and multi-copy/file integrity with minimal performance tradeoffs.

2) On-Chain Costs: For the on-chain component, we utilize Hardhat v2.22.5 for smart contract initialization, compilation, deployment, and testing, as it provides a robust and modern development environment. The smart contracts are implemented in Solidity v0.8.20 to ensure security and compatibility with current EVM standards. We employ Ethers.js v6.13.0 for interacting with the deployed contracts and scripting the test cases. The local blockchain environment is simulated using the Hardhat Network, running on a JavaScript runtime based on Node.js v20.10.0. This modern toolchain ensures that our evaluation reflects the performance and gas costs of current Ethereum networks.

As described in the scheme design, most on-chain operations involve storing data and performing basic computations. Therefore, our blockchain-side evaluation primarily focuses

on five core functionalities: random value generation, data publication, and proof verification by nodes, update performance, and batch audit scalability. It is worth noting that this study does not aim to optimize consensus mechanisms. Hence, aspects like transaction throughput and latency are excluded from our experiments.

To evaluate the overhead of generating secure randomness, we conducted unit testing on the RANDAO smart contract, performing 200 executions to measure the runtime of its core functions. As illustrated in Fig. 3(e), the time required for executing the main routines is relatively low when compared to the total cost of the auditing workflow (*ChallGen, ProofGen, ProofVerify*). Furthermore, leveraging RANDAO helps prevent manipulation by a potentially malicious TPA, thereby enhancing the security of MPA. Given these benefits, the additional gas and computation costs are considered acceptable.

Testing the gas cost provides valuable insight into the overhead of on-chain operations, which is primarily influenced by storage and computational complexity. As illustrated in Fig. 3(f), the dominant source of gas expenditure arises from persistent data storage on the blockchain. Among all operations, operation b incurs the highest gas cost due to its combination of intensive computation and substantial storage demand. In contrast, operation a consumes minimal gas, as it only writes a small, fixed-size data segment. Operations c and d contribute to blockchain size growth and thus incur moderate gas costs. Overall, the variation in gas usage reflects the differing roles and resource requirements of each operation.

3) Remark on PCS Selection: While transparent PCS such as inner product arguments (e.g., Bulletproofs) eliminate the need for a trusted setup, they are ill-suited for lightweight blockchain-based auditing. While Merkle trees are employed in our framework to enable efficient dynamic updates (introducing common logarithmic overhead), transparent alternatives additionally suffer from logarithmic proof sizes and, in many cases, linear verification complexity. This computational overhead would result in prohibitively high gas costs on the EVM. In contrast, KZG offers constant-size polynomial openings ($O(1)$) and constant-time on-chain verification via precompiled bilinear pairing contracts. By explicitly defining the trapdoor α as the user's secret key, our scheme mitigates the trust risks of the setup while retaining the superior gas efficiency required for practical decentralized auditing, as evidenced by the low gas consumption in Fig. 3(f).

MPA demonstrates clear advantages across all experimental metrics. It achieves low computation cost, acceptable on-chain cost, localized updates, and scalable auditing. Compared to prior works, MPA balances structural binding, efficiency, and multi-copy/file integrity with minimal performance tradeoffs.

4) Dynamic Update Performance: We evaluate the efficiency of dynamic operations by measuring the time required to update a single data block as the total file size (n) varies from 1,000 to 10,000 blocks. Fig. 4 compares MPA with Merkle-PDP and Edasvic. MPA demonstrates superior efficiency with an average update time of approximately 15 ms, maintaining a nearly flat curve. This validates that our rank-based Merkle tree design limits update complexity to $O(\log n)$ (hashing along the path). In contrast, Edasvic exhibits a linear growth trend, reaching over 300 ms at $n = 10,000$. This is because updating cryptographic accumulators often requires recomputing witness values for peer blocks or refreshing global parameters. MPA's position-independent polynomial tags eliminate the need for such global re-computation, making it highly suitable for dynamic decentralized storage.

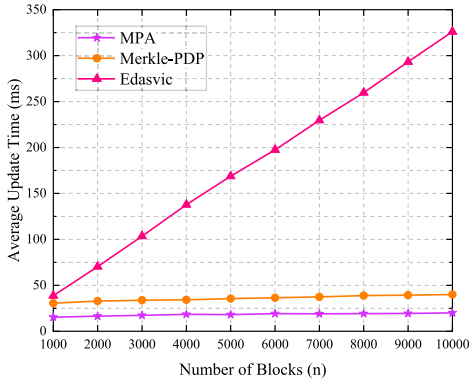


Fig. 4. Dynamic update cost.

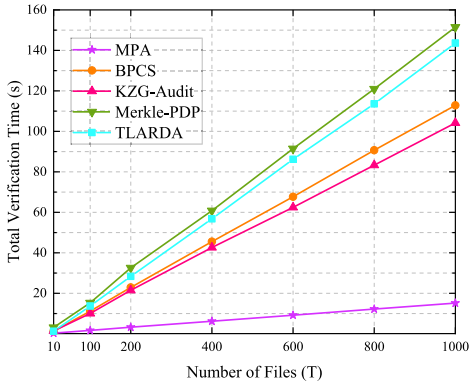


Fig. 5. Large-scale multi-file verification.

5) Scalability in Large-Scale Multi-File Auditing: To assess scalability and the impact of I/O bottlenecks, we measured the total verification time for auditing multiple files (T) ranging from 10 to 1,000, with $N = 3$ replicas fixed. As shown in Fig. 5, MPA significantly outperforms the baseline schemes (BPCS and KZG-Audit). Although the communication overhead (transmitting Merkle paths) increases linearly with T, MPA's verification time grows much slower than BPCS. For $T = 1,000$ files, BPCS requires 1,000 separate pairing checks, causing the verification time to exceed 100 seconds. MPA, leveraging the homomorphic properties of KZG, aggregates all responses into a single pairing check. While the I/O cost for reading proofs accumulates, the elimination of redundant cryptographic operations allows MPA to complete the audit in under 15 seconds. This confirms that MPA effectively mitigates computational bottlenecks in large-scale scenarios.

VIII. Conclusion

This paper presents MPA, an efficient and publicly verifiable data integrity auditing scheme for decentralized storage with multiple file copies and service providers.

By combining polynomial commitment and Merkle tree structures, MPA achieves block-level auditability, and strong binding. The use of blockchain-assisted randomness ensures unpredictability and audit freshness, while the unique encryption of each file copy prevents CSP collusion. MPA further supports aggregation across multiple files and providers, enabling a single pairing-based verification for large-scale audits. Theoretical analysis and implementation results confirm that MPA achieves correctness, unforgeability, and soundness under standard cryptographic assumptions, offering a practical and scalable solution for integrity auditing in decentralized environments.

In future work, we plan to further optimize the computational efficiency of our scheme by exploring more lightweight cryptographic primitives and parallelizable protocols. Additionally, integrating our solution with emerging technologies such as zero-knowledge proofs or post-quantum cryptography may further enhance the robustness and long-term viability of the system.

REFERENCES

[1] Y. Xu, H. Cheng, X. Liu, C. Jiang, X. Zhang, and M. Wang, "PCSE: Privacy-preserving collaborative searchable encryption for group data sharing in cloud computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 4558–4572, May 2025.

[2] Y. Xu, H. Cheng, J. Li, X. Liu, X. Zhang, and M. Wang, "Lightweight multi-user public-key authenticated encryption with keyword search," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3234–3246, 2025.

[3] International Data Corporation (IDC). (2024). *Worldwide IDC Global Datasphere Forecast, 2024–2028: AI Everywhere, but Upsurge in Data Will Take Time*. Accessed: May 3, 2025. [Online]. Available: https://www.idc.com/getdoc.jsp?containerId=US52076424

[4] Mordor Intelligence.(2025). *Cloud Storage Market-Growth, Trends, and Forecasts (2025–2030)*. Accessed: May 3, 2025. [Online]. Available: https://www.mordorintelligence.com/industry-reports/cloud-storage-market

[5] I. Jain. (2026). *Filecoin (FIL) Price Prediction 2025, 2026 and 2030*. Accessed: May 3, 2025. [Online]. Available: https://www.benzinga.com/money/filecoin-price-prediction

[6] J. Zhao, H. Huang, Y. Xu, X. Zhang, H. Du, and C. Huang, "Verifiable attribute-based multi-keyword search scheme with sensitive information hiding for cloud-assisted e-healthcare sharing systems," *Theor. Comput. Sci.*, vol. 1023, Jan. 2025, Art. no. 114895.

[7] M. Tian, C. Gao, L. Chen, H. Zhong, and J. Chen, "Proofs of retrievability with public verifiability from lattices," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3925–3935, 2025.

[8] Y. Miao, K. Gai, L. Zhu, K.-K.-R. Choo, and J. Vaidya, "Blockchain-based shared data integrity auditing and deduplication," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3688–3703, Jul. 2024.

[9] G. Tian, J. Wei, M. Miao, F. Guo, W. Susilo, and X. Chen, "Blockchain-based compact verifiable data streaming with self-auditing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3917–3930, Jul. 2024.

[10] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2007, pp. 598–609.

[11] A. Juels and B. S. Kaliski, "PORs: Proofs of retrievability for large files," in *Proc. 14th ACM Conf. Comput. Commun. Security*, New York, NY, USA, 2007, pp. 584–597.

[12] H. Shacham and B. Waters, "Compact proofs of retrievability," *J. Cryptol.*, vol. 26, no. 3, pp. 442–483, Jul. 2013.

[13] K. D. Bowers, A. Juels, and A. Oprea, "HAIL: A high-availability and integrity layer for cloud storage," in *Proc. Int. Conf. Comput. Commun. Security*, 2009, pp. 187–198.

[14] C. Zhang, H. Xuan, T. Wu, X. Liu, G. Yang, and L. Zhu, "Blockchain-based dynamic time-encapsulated data auditing for outsourcing storage," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1979–1993, 2024.

[15] Y. Du, A. Zhou, and C. Wang, "DWare: Cost-efficient decentralized storage with adaptive middleware," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8529–8543, 2024.

[16] Y. Xu, C. Jin, W. Qin, J. Shan, and Y. Jin, "Secure fuzzy identity-based public verification for cloud storage," *J. Syst. Archit.*, vol. 128, Jul. 2022, Art. no. 102558.

[17] M. Song, Z. Hua, Y. Zheng, T. Xiang, and X. Jia, "Enabling transparent deduplication and auditing for encrypted data in cloud," *IEEE Trans. Depend. Sec. Comput.*, vol. 21, no. 4, pp. 3545–3561, Apr. 2024.

[18] X. Yu, H. Bai, Z. Yan, and R. Zhang, "VeriDedup: A verifiable cloud data deduplication scheme with integrity and duplication proof," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 680–694, Jan. 2023.

[19] C. Liang et al., "Study on data storage and verification methods based on improved Merkle mountain range in IoT scenarios," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 36, no. 6, Jul. 2024, Art. no. 102117.

[20] G. Hou et al., "Cross-cloud associated multireplica auditing for lightweight devices in the IoT," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 23495–23509, Jul. 2025.

[21] Y. Miao et al., "Efficient privacy-preserving spatial range query over outsourced encrypted data," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3921–3933, 2023.

[22] C. Dong, Y. Wang, A. Aldweesh, P. McCorry, and A. van Moorsel, "Betrayal, distrust, and rationality: Smart counter-collusion contracts for verifiable cloud computing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 211–227.

[23] H. Duan, Y. Du, L. Zheng, C. Wang, M. H. Au, and Q. Wang, "Towards practical auditing of dynamic data in decentralized storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 708–723, Jan. 2023.

[24] T. Le, P. Huang, A. A. Yavuz, E. Shi, and T. Hoang, "Efficient dynamic proof of retrievability for cold storage," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2023, pp. 1–18, doi: 10.14722/ndss.2023.23307.

[25] Y. Xu, C. Jin, W. Qin, J. Zhao, G. Chen, and F. Zeng, "BDACD: Blockchain-based decentralized auditing supporting ciphertext deduplication," *J. Syst. Archit.*, vol. 147, Feb. 2024, Art. no. 103053.

[26] H. Yu, Y. Chen, Z. Yang, Y. Chen, and S. Yu, "EDCOMA: Enabling efficient double compressed auditing for blockchain-based decentralized storage," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 2273–2286, Sep. 2024.

[27] H. Yu, H. Zhang, Z. Yang, and S. Yu, "Edasvic: Enabling efficient and dynamic storage verification for clouds of industrial internet platforms," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6896–6909, 2024.

[28] A. Kate, G. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Proc. Adv. Cryptol. (ASIACRYPT)*, vol. 6477, 2010, pp. 177–194.

[29] X. Wang, X. Zhang, X. Zhang, Y. Miao, and J. Xue, "Enabling anonymous authorized auditing over keyword-based searchable ciphertexts in cloud storage systems," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4220–4232, Nov. 2023.

[30] Q. Zhang et al., "Efficient blockchain-based data integrity auditing for multi-copy in decentralized storage," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3162–3173, Dec. 2023.

[31] K. Liu, J. Ning, P. Wu, S. Xu, and R. Chen, "TLARDA: Threshold label-aggregating remote data auditing in decentralized environment," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3146–3160, 2025.

[32] D. Boneh and M. Franklin, "Identity-based encryption from the Weil pairing," *SIAM J. Comput.*, vol. 32, no. 3, pp. 586–615, Jan. 2003.

[33] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. Adv. Cryptol. (CRYPTO)*, 1988, pp. 369–378.

[34] J. Shu, X. Zou, X. Jia, W. Zhang, and R. Xie, "Blockchain-based decentralized public auditing for cloud storage," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2366–2380, Oct. 2022.

Yongliang Xu is currently pursuing the Ph.D. degree with the School of Mathematics and Statistics, Fuzhou University, Fuzhou, China. His recent research interests include applied cryptography and cloud computing security.

Hang Cheng received the B.S. and M.S. degrees in applied mathematics from Fuzhou University, Fuzhou, China, in 2002 and 2005, respectively, and the Ph.D. degree in signal and information processing with Shanghai University, Shanghai, China, in 2016. He is currently a Professor with the School of Mathematics and Statistics, Fuzhou University, Fuzhou. He is a Research Scholar with the School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore. His current research interests include multimedia security, image processing, cryptography, and information hiding.

Jingyu Zheng received the B.S. degree from Shantou University, Shantou, China, in 2024. She is currently pursuing the M.S. degree with the School of Mathematics and Statistics, Fuzhou University, Fuzhou, China. Her recent research interests include cloud computing security and searchable encryption.

Xinpeng Zhang (Senior Member, IEEE) received the B.S. degree in computational mathematics from Jilin University, China, in 1995, and the M.E. and Ph.D. degrees in communication and information systems from Shanghai University, China, in 2001 and 2004, respectively. Since 2004, he has been a Faculty Member of the School of Communication and Information Engineering, Shanghai University, where he is currently a Professor. He is also a Faculty Member of the School of Computer Science, Fudan University. He was a Visiting Scholar with The State University of New York at Binghamton from 2010 to 2011, and also with Konstanz University as an experienced Researcher, sponsored by the Alexander von Humboldt Foundation from 2011 to 2012. His research interests include multimedia security, image processing, and digital forensics. He has published over 200 articles in these areas. He has served as an Associate Editor for IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY from 2014 to 2017.

Huaxiong Wang received the Ph.D. degree in mathematics from the University of Haifa, Israel, in 1996, and the Ph.D. degree in computer science from the University of Wollongong, Australia, in 2001. He joined Nanyang Technological University in 2006. He is currently a Professor with the Division of Mathematical Sciences. He is also an honorary fellow with Macquarie University, Australia. His research interests include cryptography, information security, coding theory, combinatorics, and theoretical computer science. He has been on the editorial board of three international journals: *Designs, Codes and Cryptography* from 2006 to 2011, the *Journal of Communications* (JCM), and *Journal of Communications and Networks*. He was the program Co-chair of Ninth Australasian Conference on Information Security and Privacy (ACISP 04) in 2004 and Fourth International Conference on Cryptology and Network Security (CANS 05) in 2005, and has served in the program committee for more than 70 international conferences. He received the inaugural Award of Best Research Contribution from the Computer Science Association of Australasia in 2004.